

GROUP PROJECT

- Module: **CST2550 Software Engineering Management and Development**

Postal Service Management System

Time Complexity Analysis

→ INTRODUCTION:

- This section analyzes the time complexity of the key operations performed by each custom data structure implemented in the Postal Service Management System. **Big O notation** is used to describe the efficiency of each operation in terms of the number of elements (n) in the data structure.

→ Big O Types:

Big O	Meaning	Speed
$O(1)$	Constant time	Very fast
$O(\log n)$	Logarithmic	Fast
$O(n)$	Linear	OK
$O(n^2)$	Quadratic	Slow
$O(2^n)$	Exponential	Very Slow

1) Hash Table Analysis:

Hash Table → Parcel Storage and Search

- The Hash Table is used to store and search parcels by their unique TrackingID.

Operation	Time Complexity	Reason
Insert	$O(1)$ average	Direct index calculation
Search	$O(1)$ average	Direct index lookup
Delete	$O(1)$ average	Direct index access
Insert	$O(n)$ worst	All items hash to same bucket
Search	$O(n)$ worst	All items hash to same bucket
Delete	$O(n)$ worst	All items hash to same bucket

- The Hash Table provides $O(1)$ average time complexity for all key operations making it the most efficient data structure for searching parcels by TrackingID. The worst-case $O(n)$ occurs when all parcels hash to the same bucket causing a chain of collisions.

2) BST (Binary Search Tree) Analysis:

BST (Binary Search Tree) → Sorted Parcel Views

- The BST is used to store parcels in sorted order by TrackingID for sorted views and range searches.

Operation	Time Complexity	Reason
Insert	$O(\log n)$ avg	Halves search space
Search	$O(\log n)$ avg	Halves search space
Delete	$O(\log n)$ avg	Halves search space
IN Order Traverse	$O(n)$	Visits every node once
Insert	$O(n)$ worst	Unbalanced tree
Search	$O(n)$ worst	Unbalanced tree

- The BST provides $O(\log n)$ average time complexity for insert and search operations. The worst-case $O(n)$ occurs when the tree becomes unbalanced. For example, when parcels are inserted in alphabetical order causing the tree to become a straight line.

3) Queue Analysis:

Queue → Delivery Management

- The Queue is used to manage the delivery pipeline following First in First Out (FIFO) order.

Operation	Time Complexity	Reason
Enqueue	O (1)	Always adds to rear
Dequeue	O (1)	Always removes from front
Peek	O (1)	Just reads front node
Is Empty	O (1)	Just checks front node
Get Size	O (1)	Maintains a count variable

- The Queue provides O (1) time complexity for all operations because it always operates on the front or rear of the queue without traversing the entire structure.

Data Structure Selection Justification:

Hash Table

- The Hash Table was selected as the primary data structure because the most frequent operation in a postal system is searching for a parcel by its unique TrackingID. The Hash Table provides O (1) average search making it the fastest possible choice for this operation.

BST (Binary Search Tree) Analysis:

- The BST was selected to support sorted views of parcels by TrackingID or date. Its $O(\log n)$ operations make it efficient for ordered data retrieval which the Hash Table cannot provide.

Queue Analysis:

- The Queue was selected to manage the delivery pipeline because deliveries must be processed in the order they were assigned, First in First Out. The Queue provides $O(1)$ enqueue and dequeue operations making it perfectly suited for this purpose.